

Exploring Artificial Neural Networks for Fashion MNIST Classification (July 2024)

Aayush Man Shrestha¹ and Pradeepti Dongol²

Department of Electronics and Computer Engineering, IOE, Thapathali Campus, Kathmandu 44600, Nepal

Corresponding author: Aayush Man Shrestha (aayush.m.shrestha@gmail.com)

ABSTRACT Artificial Neural Networks (ANNs) offer powerful capabilities for complex data classification tasks. This report explores the application of ANNs to the Fashion MNIST dataset, which consists of images of clothing items. We investigate the impact of varying the number of hidden layers, different weight initialization mechanisms (Random, Kaiming, and Xavier), and activation functions (ReLU, Tanh, and Sigmoid) on the model's classification performance. The study reveals that increasing the number of hidden layers improves accuracy and stability in learning, with Tanh activation function providing the highest performance due to its smooth gradient properties. Additionally, the effect of dropout as a technique to mitigate overfitting is examined, demonstrating enhanced generalization and performance stability.

INDEX TERMS Activation Functions, Artificial Neural Networks, Dropout, Fashion MNIST, Overfitting, Weight Initialization

I. INTRODUCTION

Artificial Neural Networks (ANNs) have emerged as a cornerstone of modern machine learning, demonstrating remarkable capabilities in various domains, including image recognition, natural language processing, and predictive analytics. Inspired by the structure and function of the human brain, ANNs consist of interconnected nodes, or neurons, that work collaboratively to solve complex problems by learning from data. ANNs excel at learning complex patterns and making predictions from data. This ability has made them indispensable tools in various fields, including image and speech recognition, natural language processing, and financial forecasting. This lab report focuses on applying ANNs to the Fashion MNIST dataset, a benchmark dataset consisting of 70,000 grayscale images of 10 distinct categories of fashion items. By understanding the underlying theory and conducting experiments, we aim to gain insights into the capabilities and limitations of ANNs in classifying fashion items accurately.

Mathematical Model of Neurons

Inputs: A set of numerical values, denoted as $x_1, x_2, \dots, x_n$, representing the features of the data.

Weights: Numerical values associated with each input, denoted as $w_1, w_2, \dots, w_n$. These weights determine the importance of each input in the computation.

Bias: A constant value, denoted as b , added to the weighted sum to adjust the output. It allows the activation function to be shifted.

Weighted Sum: The sum of the products of each input and its corresponding weight, plus the bias. Mathematically, it is expressed as:

$$z = w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n + b.$$

Activation Function: A non-linear function applied to the weighted sum z to introduce non-linearity into the model, making it capable of learning complex patterns and relationships in the data. The output of the neuron is given by: $y=f(z)$ Without activation functions, a neural network would behave like a linear model, limiting its ability to capture intricate patterns. Common activation functions include sigmoid, tanh,

ReLU (Rectified Linear Unit) and Leaky ReLU.

An ANN typically consists of multiple layers:

Input Layer: The input layer is the first layer of the neural network. Its primary function is to receive raw data from the external environment. This data can be in various forms such as images, text, or numerical values.

Hidden Layers: The hidden layers are intermediate layers between the input and output layers. They process the input data by performing complex transformations and feature extraction. These layers are called "hidden" because they are not directly exposed to the input or output but work behind the scenes. Each hidden layer consists of multiple neurons, each receiving input from the previous layer. The number of hidden layers and the number of neurons in each layer can vary depending on the network architecture and the complexity of the problem.

Output Layer: The output layer is the final layer of the neural network. It produces the final result or prediction based on the transformed data from the hidden layers. The number of neurons in the output layer corresponds to the number of possible output classes.

Neurons in each layer are connected to neurons in adjacent layers through weighted connections. These weights determine the strength and influence of the connection between neurons. During the training process, the network learns the optimal weights for these connections to minimize the error in predictions. The learning process involves adjusting the weights based on the input data and the error feedback.

Learning Process

Artificial Neural Networks (ANNs) learn through a process called backpropagation, which systematically adjusts the network's weights to minimize the error in its predictions. This process can be broken down into several key steps:

Forward Propagation: The learning process begins with forward propagation, where input data is fed through the network starting from the input layer. This data moves sequentially through each hidden layer and ultimately reaches the output layer. During this journey, each neuron in the hidden layers computes a weighted sum of its inputs, adds a bias term, and applies an activation function to produce its output.

A. Shrestha and P. Dongol: Artificial Neural Network

The final layer, the output layer, uses these transformed inputs to produce the network's prediction.

Error Calculation: Once the network generates a prediction, the next step involves calculating the error or loss, which measures the difference between the predicted output and the actual output (ground truth). This error quantifies how well the network's prediction aligns with the true values.

Loss Function: The error is computed using a loss function, which is crucial for guiding the learning process. Common loss functions include Mean Squared Error (MSE) for regression tasks and Cross-Entropy Loss for classification tasks. These functions provide a mathematical framework for quantifying the prediction error, allowing the network to understand how far off its predictions are from the actual values.

Backpropagation: After calculating the error, the network uses backpropagation to propagate this error backward through the network. This step starts at the output layer and moves backward to the input layer. During backpropagation, the network computes the gradient of the loss function with respect to each weight. These gradients indicate the direction and magnitude of change needed for each weight to reduce the overall error.

Gradient Descent: The computed gradients are then used in an optimization algorithm such as gradient descent. Gradient descent adjusts the weights in the opposite direction of the gradients to minimize the error. This process involves small, iterative steps, gradually moving the weights towards values that reduce the prediction error.

Weight Update: In each iteration, or epoch, the weights are updated based on the gradients. The amount by which the weights are adjusted is controlled by a learning rate, a hyperparameter that determines the size of the steps taken during optimization. This iterative adjustment continues until the network reaches a state where the error is minimized, and the predictions are as accurate as possible.

II. RELATED WORK

Fashion MNIST serves as a more complex drop-in replacement for the original MNIST dataset of handwritten digits, providing a more challenging task for

image classification algorithms while still allowing for a comparison of results across different models and approaches [1].

Feedforward Neural Networks (FNNs) are the simplest type of artificial neural network architecture, where the connections between nodes do not form a cycle. In an FNN, data moves in one direction, from input nodes through hidden nodes (if any) to output nodes. Despite the rise of more complex architectures like Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), FNNs have been extensively studied and applied to various classification tasks, including image classification.

Although FNNs are generally less suited for image classification tasks compared to CNNs due to their inability to capture spatial hierarchies, several studies have explored their application to the Fashion MNIST dataset. For instance, a study by Li et al. utilized a simple three-layer FNN for classifying Fashion MNIST images and achieved reasonable accuracy by optimizing hyperparameters and incorporating regularization techniques [2].

Hyperparameter optimization plays a crucial role in enhancing the performance of FNNs. Techniques such as grid search, random search, and more advanced methods like Bayesian optimization have been employed to find the optimal set of hyperparameters for FNNs. Bergstra and Bengio demonstrated that random search can be more efficient than grid search in hyperparameter optimization, particularly when dealing with high-dimensional spaces [3].

Regularization techniques, such as L2 regularization and dropout, are essential for preventing overfitting in FNNs. Srivastava et al. introduced dropout as a method to improve the generalization of neural networks by randomly dropping units during training. This technique has been effectively applied to FNNs to enhance their performance on the Fashion MNIST dataset [4].

The choice of activation functions significantly impacts the learning and performance of FNNs. Commonly used activation functions include the sigmoid, hyperbolic tangent (tanh), and rectified linear unit (ReLU). The ReLU activation function, in particular, has gained popularity due to its ability to mitigate the vanishing gradient problem and accelerate the training of deep networks [5].

A. Shrestha and P. Dongol: Artificial Neural Network

Batch normalization is another technique that has been shown to improve the performance of FNNs. Ioffe and Szegedy introduced batch normalization to address the issue of internal covariate shift, thereby stabilizing and accelerating the training process. This technique has been successfully applied to FNNs to enhance their robustness and classification accuracy on various datasets, including Fashion MNIST [6].

Comparative studies have highlighted the strengths and weaknesses of FNNs in the context of image classification. For example, a study by Zhang et al. compared the performance of FNNs with and without batch normalization on the Fashion MNIST dataset, demonstrating that batch normalization significantly improves the model's accuracy and convergence speed [7].

III. METHODOLOGY

A. DATASET DESCRIPTION

Fashion MNIST dataset, from Zalando's article images, consists of 70,000 images that are divided into two sets: 60,000 training data images and 10,000 test data images. Every image in the dataset is associated with its corresponding class label.

Each image in the dataset is a 28x28 grey-scale image where each pixel value represents the lightness and darkness of that pixel. The pixel values are represented by integers ranging from 0 to 255 where higher pixel values represents darker colour.

The 28x28 pixel images are represented by a row matrix of 784 columns, where each column represents a pixel value. The dataset, however, consists of 785 columns where the first column represents the class label while the remaining 784 columns represent the pixel values of the corresponding image.

Since each image is represented as a row matrix containing 785 columns, each instance represents an image in the dataset. Hence, the dataset consisting of 70,000 rows, each representing one of the 70,000 images.

The dataset consists of 10 classes where, each class represents an article of clothing. The class value is represented by an integer in the range 0 to 9, with each integer corresponding to a specific class value.

The classes of the Fashion-MNIST dataset are:

Class 0: T-shirt

Class 1: Trouser

Class 2: Pullover
Class 3: Dress
Class 4: Coat
Class 5: Sandal
Class 6: Shirt
Class 7: Sneaker
Class 8: Bag
Class 9: Ankle boot

B. PROPOSED METHODOLOGY

Data Preprocessing: The preprocessing stage of developing an Artificial Neural Network (ANN) involves various steps such as imputing missing values, handling categorical data, discretizing, treating outliers, normalizing, standardizing and rectifying class imbalance.

Since the Fashion MNIST dataset is a standard dataset containing 28x28 pixel images along with its class label, it does not contain any missing values. So, imputation of missing values need not be performed. The dataset contains 784 features, each of integer data type, and one categorical label. So, the label should be encoded using one hot encoding method. Due to the absence of continuous data, discretization is not necessary.

The feature value ranges from 0 to 255, each value representing the intensity or shade of grey of each pixel. Hence the data doesn't contain extreme or unexpected values which ensures that outliers are effectively absent.

Since using a large range of integers from 0 to 255 can create problems, such as dominance of larger numerical values, difficult optimization, risks of vanishing gradients and poor performance of artificial neural network, the 784-pixel values are normalized to a range of 0 to 1 by dividing by 255. The Fashion MNIST dataset is a machine learning benchmark dataset. Thus, each image of the dataset is distributed to the 10 target classes with roughly equal frequency resulting in a uniformly distributed dataset allowing models to be trained and evaluated effectively without biases.

Initialization of Model Parameters: The parameters such as weights and biases are initialized in an artificial neural network using multiple methods. Some of which are Random Initialization Kaiming Initialization and Xavier or Glorot Initialization. These three initializations were defined in the model using `randomInitialization()`, `kaimingInitialization()` and `xavierInitialization()` functions.

A. Shrestha and P. Dongol: Artificial Neural Network

Initially the parameters of the ANN model are initialized using Random Initialization. The weights are initialized by selecting a random value from the range -0.5 to 0.5 while the biases are initialized to zeros. This method ensures that each neuron starts with different initial parameters to help the model to break the symmetry and ensure efficient learning. The weights and biases are vectors corresponding to the size of the neural network layers that are connected to them.

Kaiming Initialization or He Initialization is particularly implemented in neural networks using rectified linear unit (ReLU) activation function or for variants of ReLU. In this method of initialization, the weights are generated randomly from a normal distribution with mean zero and are scaled by a factor that helps to maintain the variance of the activations constant across layers while the biases are initialized to zero.

The third type of initialization implemented is Xavier Initialization. It aims to maintain variance of activation and gradients throughout the layers of a deep neural network to achieve better convergence during training and mitigates issues related to vanishing and exploding gradients. This method randomly generates weights from a distribution where the variance is inversely proportional to the number of input and output units of the layer. The values of biases are always initialized to zero.

Activation Functions: In ANN (Artificial Neural Network), activation functions are mathematical functions that introduce non-linearity to the neural network so that the model is capable of learning and representing complex relationships between inputs and outputs. Activation functions are applied to each neuron's output before passing it to the next layer. Among various activation functions, this report utilizes Rectified Linear Unit (ReLU), Leaky ReLU, Hyperbolic Tangent (tanh), Sigmoid and Softmax activation functions as `ReLU(vector)`, `leakyReLU(vector)`, `tanh(vector)`, `sigmoid(vector)` and `softmax(vector)`.

ReLU (Rectified Linear Unit) is a piecewise linear activation function that ranges from 0 to infinity. It outputs zero if the input is a negative number otherwise it outputs the number itself.

The Leaky ReLU activation function is a variation of ReLU function with a small non-zero gradients for negative input to

address the dying ReLU problem. It ranges from negative infinity to positive infinity. The sigmoid activation function is an S-shaped function ranging between 0 and 1 that models probabilities. It is often used in the output layer of binary classification problems.

The Hyperbolic Tangent (tanh) activation function ranges from -1 to 1 and represents a sigmoid-shaped function that outputs zero-centered values assisting in faster convergence.

Softmax activation function is a generalization of the sigmoid function that outputs probability distribution over multiple classes. It ranges from 0 to 1 and the sum of outputs equal to 1.

The ReLU, Leaky ReLU, Sigmoid and Hyperbolic tangent functions were used in the hidden layer of the proposed ANN while the softmax was used in the output layer.

The derivative of these activation functions is defined and used for backpropagation.

Forward Propagation: Forward propagation includes passing values from one layer to another by performing operations and applying activation functions.

Initially, the input data (X) is passed into the networking. The linear transformation of each layer (Z) is computed using input, weight and bias of the layer. Similarly, the activation (A) of each layer is obtained by applying activation function to the linear transformation output of that layer. This process is repeated for all the layers of the neural network until an output is produced. Forward propagation is used to compute the output predictions and training loss of the network.

Backward Propagation and Gradients: Back propagation involves computing gradients of loss function with respect to the model parameters like activations, pre-activation values, weights and bias of each layer. The results obtained from backpropagation is passed on to update parameters.

Update Parameters: This step involves adjusting the weights and biases based on the gradients computed during backward propagation and learning rate. The weights and biases are updated by subtracting the product of learning rate and gradient of corresponding parameter from the current parameter value.

Using Dropout: To prevent overfitting of the model, dropout is used. Dropout randomly sets a portion of the neurons to zero

during training which improves the model's ability to generalize to unseen data.

Based on a predefined dropout rate, a binary mask is created that drops neurons that exceed the required threshold or apply mask to the activations setting some neurons to zero. The result is scaled by dividing by '1-dropout' rate to maintain expected value. The results obtained are utilized during forward propagation and backward propagation.

Prediction Generation: Using the `np.argmax` function, the label of test data are predicted.

Accuracy Calculation: The predicted label is compared with the reference label to determine the accuracy of the model.

Gradient Descent Iterations: Gradient descent includes iteratively updating weights and biases while reducing the loss and improving model's performance to train the artificial neural network model.

Testing and Visualization: Once trained, the performance of the model is tested using loss vs iterations curve, accuracy vs iterations curve, confusion matrix and classification report

Algorithm: ANN from scratch

1. Load train and test dataset
 2. Check for missing values
 3. Check data distribution to confirm uniform distribution
 4. Convert data to NumPy array and shuffle the data
 5. Transpose data to represent each image by a row
 6. Normalize data by dividing pixel values by 255
 7. Split data into features (X) and labels (y)
 8. Perform one hot encoding label
 9. Initialize using Random Initialization, Kaiming Initialization and Xavier Initialization to specify weights and biases between neurons
 10. Define activation functions including ReLU, Softmax, Hyperbolic tangent, Sigmoid and their derivatives
 11. Define a function to create a dropout mask and apply and scale the activations
 12. Define forward propagation using chosen activation functions and dropout mask
 13. Define backward propagation using gradients of loss function, derivatives of the activation functions and dropout mask
-

14. Define parameter update function using learning rate and corresponding gradients of loss function
15. Define prediction function to predict the labels of test data
16. Calculate accuracy of predicted labels with respect to reference labels
17. Implement gradient descent algorithm
18. Evaluate the model using accuracy vs iteration graphs, loss vs iteration graphs, confusion matrix and classification report

C. MATHEMATICAL FORMULAE

Kaiming Scaling Factor: The scaling factor of Kaiming Initialization is:

$$\text{Kaiming scaling factor} = \sqrt{\frac{2}{n_{in}}} \quad (1)$$

where, n_{in} represents the number of input units.

Xavier Scaling Factor: The scaling factor for Xavier Initialization is:

$$\text{Xavier scaling factor} = \sqrt{\frac{2}{n_{in} + n_{out}}} \quad (2)$$

where, n_{in} and n_{out} represents the number of input and output units respectively.

Mathematical Model of Neuron: The neuron computes a weighted sum of the inputs and adds bias to it.

$$z = \sum_{i=1}^n w_i x_i + b \quad (3)$$

where, n is the number of inputs, w_i is the weight of input x_i and b is the bias term.

The weighted sum z is passed through an activation function $\phi(x)$.

ReLU Activation Function: The ReLU activation function is

$$\phi(x) = \begin{cases} 0 & x \leq 0 \\ x & x \geq 0 \end{cases} \quad (4)$$

The derivative of ReLU activation function is

$$\phi'(x) = \begin{cases} 0 & x \leq 0 \\ 1 & x \geq 0 \end{cases} \quad (5)$$

Hyperbolic Tangent Activation Function: The hyperbolic tangent activation function is

$$\phi(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1 \quad (6)$$

A. Shrestha and P. Dongol: Artificial Neural Network

The derivative of hyperbolic tangent activation function is

$$\phi'(x) = 1 - \phi(x)^2 \quad (7)$$

Sigmoid Activation Function: The sigmoid activation function is

$$\phi(x) = \frac{1}{1 + e^{-x}} \quad (8)$$

The derivative of sigmoid activation function is

$$\phi'(x) = \phi(x)(1 - \phi(x)) \quad (9)$$

Softmax Activation Function:

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (10)$$

where, σ , $\vec{z}$, e^{z_i} , K and e^{z_j} represents softmax function, input vector, standard exponential function for input vector, number of classes in the multi-class classifier and standard exponential function for output vector respectively.

Cross-Entropy Loss:

The cross-entropy loss is

$$L = - \sum_{j=1}^K \hat{y}_j \log(\hat{y}_j) \quad (11)$$

Accuracy:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (12)$$

where, TP (True Positives) represents correctly predicted positive instances, TN (True Negatives) represents correctly predicted negative instances, FP (False Positives) represents incorrectly predicted positive instances and FN (False Negatives) represent incorrectly predicted negative instances.

Precision:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (13)$$

Recall:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (14)$$

F1-score:

$$F1\text{score} = 2 \times \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} \quad (15)$$

where, Precision is the ratio of correctly predicted positive instances to the total predicted positive instances and Recall ratio represents the correctly predicted positive instances to the total actual positive instances.

D. INSTRUMENTATION DETAILS

The analysis and visualization in this study were conducted using **Jupyter Notebook**, a versatile and interactive web-based coding environment that allows for the integration of code execution, rich text, mathematics, plots, and rich media in a single document. This platform was chosen for its ease of use and its extensive support for data science and machine learning workflows. The core libraries used in this study were NumPy, Pandas, Scikit-learn, Matplotlib, and Seaborn. **NumPy** is a fundamental library for scientific computing in Python, providing support for large multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays. In this study, NumPy was used extensively in various stages of developing an Artificial Neural Network (ANN) from scratch.

The **'zeros'** and **'arrange'** functions were used during one-hot encoding.

During weight initialization, functions such as **'random.seed'**, and **'random.randn'**, **'random.uniform'** were used to generate random seeds and numbers from uniform and standard normal distributions. Similarly, the **'zeros'** function was utilized to initialize biases.

In forward propagation, functions like **'dot'** were employed to perform matrix multiplications while for activation, the **'maximum'** function was used for ReLU activation, **'array'** and **'tanh'** functions were used for Hyperbolic Tangent activation, **'exp'** function was used for sigmoid activation while **'exp'** and **'sum'** were crucial for implementing the softmax activation function.

During backward propagation, **'dot'** function was used for gradient computations, and element-wise multiplications were performed with the help of **'sum'**, **'reshape'** and other basic arithmetic operations.

Loss computation involved the use of cross entropy loss functions to calculate the cross-entropy loss.

During dropout, functions such as **'random.rand'** was used to generate random numbers.

Finally, parameter updates were carried out using basic arithmetic operations on NumPy arrays.

This extensive use of NumPy allowed for efficient data manipulation and mathematical computations throughout the neural

A. Shrestha and P. Dongol: Artificial Neural Network

network's lifecycle, from initialization and training to evaluation.

Pandas is essential for data manipulation and analysis, offering data structures and functions needed to manipulate structured data seamlessly. Pandas was utilized for importing the dataset, handling missing values, and performing Exploratory Data Analysis (EDA). The library's DataFrame structure was particularly useful for organizing the data and preparing it for analysis.

Scikit-learn, a robust machine learning library in Python, provides simple and efficient tools for data mining and data analysis. It was used extensively in this study.

Evaluation metrics including **accuracy_score**, **precision_score**, **recall_score**, **f1_score**, **confusion_matrix** and **classification_report** was used to calculate and assess the performance of the models.

For data visualization, **Matplotlib** and **Seaborn** were employed. These libraries were used to create visual representations of the confusion matrix, aiding in the interpretation of the classifier's performance, and to generate histogram plots to visualize the distribution of data and results.

IV. EXPERIMENTAL RESULTS

Data Distribution: The train dataset was found to be uniformly balanced on all 10 classes with 6000 images in each class as illustrated in [Figure 1](#). Similarly, the test data was also uniformly balanced with 1000 images in each class as shown in [Figure 2](#).

Effect of increasing hidden layers: The model was evaluated two variations in number of hidden layers, using one hidden layer and using two hidden layers, to understand the effects of changing hidden layers. The first model contained one hidden layer. This model consisted of an input layer of size 784 neurons (corresponding to the 784 features or pixels), a hidden layer of size 10 neurons and an output layer of size 10 neurons (corresponding to the 10 classes in the dataset). Weights W1 and W2 were used between input and hidden layer and hidden and output layer respectively. Similarly, the biases used for hidden layer and output layer were represented by b1 and b2 respectively. The linear transformations representing input to hidden layer and output layer are Z1 and Z2 respectively while the activations representing output of hidden layer and output of network are A1 and A2 respectively.

Similarly, the second model was developed using one input layer of size 784 neurons, two hidden layers of sizes 128 and 64 neurons and an output layer of size 10 neurons. It consisted weights W_1 (from input to first hidden layer), W_2 (from first hidden layer to second hidden layer) and W_3 (from second hidden layer to output layer), biases b_1 (for first hidden layer), b_2 (for second hidden layer) and b_3 (for output layer), linear transformation Z_1 (input to first hidden layer), Z_2 (input to second hidden layer) and Z_3 (input to output layer) and activation A_1 (output of first hidden layer), A_2 (output of second hidden layer) and A_3 (output of the network).

Both the models were trained using Random initialization, ReLU activation function in the hidden layers and Softmax activation function in the output layers for 500 iterations and were finally evaluated using Accuracy Curve (Figure 3 and Figure 6), Loss Curve (Figure 4 and Figure 7), Confusion matrix (Figure 5 and Figure 8) and classification report (Table 1 and Table 2). For the first model, consisting of one hidden layer, the accuracy curve shows a significant increase in accuracy from 0.1 to 0.7 in the first 100 iterations. The accuracy briefly decreases for a few iterations and starts rising gradually increases with increase in iteration resulting in a final accuracy of 0.76, represented in the classification report. Conversely, the loss curve indicates a significant decrease in loss from 2.75 to around 0.9 in the first 100 iterations. The curve briefly increases in the iterations corresponding to brief decrease in accuracy after which the loss gradually decreases. The confusion matrix shows high diagonal values and low off-diagonal values representing that most of the images were classified correctly with the highest accuracy of 903 for class 1 (Trouser) and lowest accuracy for class 6 (Shirt). Similarly, the highest misclassification was observed to be 335 where class 4 (Coat) was predicted as class 2 (Pullover). The classification report shows that class 6 has the lowest precision, recall and f1-score values ranging from 0.33 to 0.55 while the highest values were observed for classes 1 and 8 ranging from 0.90 to 0.95. The accuracy, macro average and weighted average of the model was observed to be 0.76 (while its test accuracy was 0.8167.)

For the second model, that consisted of two hidden layers, the classification report

shows that classes 1 and 8 have the highest precision, recall and f1-score values ranging from 0.91 to 0.96 while the lowest values were observed for class 6 in the range 0.49 to 0.59. The accuracy, macro average and weighted average of the model was observed to be 0.8 (while its test accuracy was 0.7981). The accuracy curve of the model initially shows significant increase in accuracy value from 0.1 to around 0.69 after which the accuracy briefly decreases to around 0.65 for a few iterations. Following this, the accuracy gradually increased to 0.8 at the 500th iteration which is also the overall accuracy of the model. Conversely, the loss curve shows a significant decrease in loss value from around 36 to around 2 initially after which later stabilizes. The confusion matrix shows that most of the images were classified correctly with maximum values in the diagonals. The highest accuracy of 947 was obtained for class 1 while the highest misclassification was 230 where image belonging to class 4 was predicted as class 6 image.

Effect of Initialization Mechanism and Activation Functions

The ANN model containing two hidden layers was subsequently trained using three different initialization mechanisms: Random Initialization, Kaiming Initialization and Xavier Initialization. Also, each initialization mechanism was trained by changing the activation function of hidden layers using three activation functions: ReLU, Hyperbolic Tangent and Sigmoid activation functions while constantly using Softmax activation function as the output layer activation function

Initially, the model used random initialization method along with ReLU activation function to compare the model with ANN model containing lower number of neurons. Following that, the model was initialized using Random Initialization and trained using tanh activation function for hidden layers and evaluated using accuracy curve (Figure 9), loss curve (Figure 10), confusion matrix (Figure 11) and classification report (Table 3). This model produced smooth accuracy and loss curves where the accuracy significant increase in the first few iterations from 0 to 0.6 after which the increase was gradual resulting in an accuracy of more than 0.8 at the end of the iterations while loss curve experienced similar trends with significant decrease initially and gradual decrease after a few iterations. The

curves did not experience any fluctuations during training. The confusion matrix represents that the misclassification is negligible compared to the correct classification with highest misclassification of 192 where image belonging to class 6 was predicted as class 0 while the highest and lowest correct classification values were 940 for class 1 and 494 for class 6 respectively. The model exhibited accuracy, macro avg, weighted avg of 0.8 with highest and lowest precision, recall and f1-score for class 1, in the range of 0.94 to 0.96, and class 6, ranging from 0.49 to 0.60, respectively.

The model was again trained using Random Initialization with Sigmoid activation function for the hidden layers. The model was then evaluated using accuracy curve (Figure 12), loss curve (Figure 13), confusion matrix (Figure 14) and classification report (Table 4). The accuracy curve for this method showed that the increase in accuracy was significant during the first few iterations where the accuracy increased from 0 to around 0.5. After that, the rate of increase in accuracy decreased a little resulting in increase in accuracy from 0.5 to 0.7 over 200 iterations after which the increase in accuracy was very gradual. Conversely, the loss decreased from around 2.75 to 2.25 in the first few iterations. However, the rate of decrease in loss slowed down after that resulting in a decrease of loss from 2.25 to 0.75 over the next 400 iterations. The confusion matrix suggested that the model mostly classified images correctly. Classes 1, 8 and 9 were more likely to be classified correctly with 927, 905 and 914 correctly identified images out of 1000. However, the model was only capable of identifying 299 images as class 6 correctly while 223, 222 and 148 images belonging to class 6 were classified as class 4, class 0 and class 2 respectively. The classification report indicated that the model has an accuracy of 0.76. Class 1 has the best performance with precision 0.96, recall 0.93 and 0.94 while class 6 indicated poor performance with precision 0.49, recall 0.3 and f1-score 0.37. After training the model using Random Initialization, the same process was repeated for Kaiming Initialization. Initially, Kaiming Initialization was used with ReLU activation function for hidden layers. The model was evaluated using accuracy curve (Figure 15), loss curve (Figure 16), confusion matrix (Figure 17) and classification report (Table 5). The loss and accuracy

curve obtained from this model was not smooth. The loss significantly decreased in the first few iterations after which the model observed gradual decrease in loss with major fluctuation in the beginning and with the decrease in loss, the fluctuations also lessened. Similarly, the accuracy increased significantly in the first few iterations after which the model observed gradual increase in accuracy with fluctuations throughout the training process. The confusion matrix obtained for this model indicates that the model mostly identifies images correctly with 881 to 971 correct classifications out of 1000 for classes 0, 1, 2, 3, 5, 7, 8 and 9. However, classes 4 and 6 struggle with correct classification where class 4 has 676 correct classification and 240 images were misclassified as class 2 and 338 images were classified correctly as class 6 however 270 and 232 images were misclassified as class 0 and class 2 respectively. The classification report indicated that the overall accuracy of the model was 0.83 with the highest performance of precision 0.96, recall 0.97 and f1-score 0.96 of class while the recall value for class 6 was observed to be 0.34 indicating poor performance.

Next, the model was evaluated using Kaiming Initialization and Hyperbolic tangent activation function for hidden layers and evaluated using accuracy curve (Figure 18), loss curve (Figure 19), confusion matrix (Figure 20) and classification report (Table 6). The loss curve for this model was smooth with significant decrease of loss from around 2.35 to 0.75 in the first few iterations after which the decrease in loss was gradual with a loss of around 0.4 in the 500th iteration. The accuracy curve of the model indicated significant increase in accuracy from 0.2 to around 0.65 in the first few iterations. However, the increase in accuracy after that was very gradual and contained minor fluctuations throughout the process with an accuracy of about 0.85 during the 500th iteration. The confusion matrix indicates that the model was about the correctly classify most images but struggled to classify some images of class 6 resulting in class 6 being correctly classified 437 times out of 1000 and classified as class 0, 2 and 4 192, 145 and 155 times respectively. The classification report indicates that the model performs with an accuracy of 0.85 with class 1 performing the best with precision, recall and f1-score at 0.96 while

class 6 struggled with recall 0.44, precision 0.78 and f1-score 0.56.

After that, the model was trained and evaluated using Kaiming Initialization and Sigmoid activation function and the model was evaluated using accuracy curve (Figure 21), loss curve (Figure 22), confusion matrix (Figure 23) and classification report (Table 7). The loss value decrease from 2.75 to 2.25 initially in the first few iterations after which the value remains stable for a few iterations. After that, the loss decreases gradually with high rate of decrease at first.

The accuracy curve shows that the accuracy of the model increases significantly for the first 50 iterations from 0.1 to 0.6 after which the increase in accuracy from 0.6 to 0.72 is gradual over 450 iterations. Based on the confusion matrix, it is observed that classes 1, 3, 7, 8 and 9 were mostly classified correctly with a few misclassifications. Classes 0, 2 and 5 showed correct classification in the range of 651 to 771 while classes 4 and 6 contained significant misclassification. Class 6 contained 202 correctly classified images while 250, 255 and 189 images belonging to class 6 were predicted as class 0, 2 and 4 respectively. The classification report indicated an overall accuracy of 0.72. Class 1 and 8 performed well with precision, recall and f1-score ranging from 0.89 to 0.95 while class 6 indicated poor performance with precision 0.43, recall 0.2 and f1-score 0.27.

The model was again trained and evaluated using Xavier Initialization with different activation functions in the hidden layers and the models were evaluated using accuracy curve (Figure 24, Figure 27 and Figure 30), loss curve (Figure 25, Figure 28 and Figure 31), confusion matrix (Figure 26, Figure 29 and Figure 32) and classification report (Table 8, Table 9 and Table 10).

First, the model used Xavier Initialization and ReLU activation function. On doing so, the model observed that the loss initially decreased significantly from around 2.3 to 1 over the first few iterations and the rate of decrease in loss became gradual where the loss decreased from 1 to 0.5 over the next 475 iterations. Conversely, the accuracy curve depicted that the accuracy increased considerably in the first few iterations from 0.05 to 0.7 after which the increase was gradual with an accuracy of 0.84 in the 500th iteration. Both the curves contained fluctuation in loss and accuracy values throughout the training period. From the

confusion matrix, it is evident that the model was capable of identifying most images correctly. However, class 6 contained significant misclassification with 233 and 166 images predicted as class 0 and class 2 respectively and 440 images classified correctly. The classification report indicates that the model performs with accuracy, macro average and weighted average of 0.84 with highest and lowest precision, recall and f1-score given by class 1 (0.96, 0.97, 0.96) and class 6 (0.77, 0.44 and 0.56) respectively.

Next, the model was trained using Xavier Initialization and Hyperbolic tangent activation function. The model was evaluated using loss curve (fig), accuracy curve, confusion matrix and classification report. The loss curve indicated significant decrease in loss from about 0.3 to 0.75 in the first few iterations after which the loss decreased to about 0.4 in the 500th iteration. Conversely, the accuracy curve shows considerable increase in accuracy from 0.1 to 0.7 in the first few iterations after which the accuracy increases gradually with the accuracy value 0.84 at the 500th iterations. Similarly, the confusion matrix suggests that the model is capable of correctly classifying most images correctly. The classification report shows that the model performs with an accuracy of 0.84. Class 1 shows the best performance with precision 0.96, recall and f1-score 0.97 while class 6 with precision 0.63 and recall and f1-score 0.62 indicate relatively poor performance.

Finally, the model Xavier Initialization and Sigmoid activation function was used to train the model and was evaluated using accuracy curve, loss curve, confusion matrix and classification report. From the accuracy curve, it was observed that the accuracy increases substantially for the first few iterations after which a slight decrease in accuracy value is observed. Then, the accuracy begins to rise gradually. Conversely, the loss value decrease from 2.75 to 2.25 initially in the first few iterations after which the value remains stable for a few iterations. After that, the loss decreases gradually with high rate of decrease at first. Similarly, from the confusion matrix, it is observed that classes 1, 3, 7, 8 and 9 are classified correctly with high accuracy with 923, 809, 844, 893 and 913 correct classifications out of 1000 while 774, 562, 548 and 572 images are classified correctly as class 0, 2, 4 and 5 indicating moderate accuracy.

However, for class 6 the number of misclassified images is greater than the number of correctly classified images (209) indicating poor performance.

Mitigating Overfitting (Using Dropout)

To study the effects of overfitting, dropout was used on two models: Random Initialization with ReLU activation function in hidden layers and Softmax activation function in output layer, and Kaiming Initialization with Hyperbolic Tangent activation function in hidden layers and Softmax activation function in output layer. Both the models were evaluated using accuracy curve (Figure 33 and Figure 36), loss curve (Figure 34 and Figure 37), confusion matrix (Figure 35 and Figure 38) and classification report (Table 11 and Table 12).

For the first mode, the accuracy increased drastically for the first few iterations after which it showed fluctuation once and then continued to increase in a steady pace to achieve an accuracy of 0.81 by the end of the training process. The loss curve, however represented steep decline in loss value resulting in almost minimum loss value in the first few iterations after which the curve appears to be declining at a rate at which the curve appears to be almost constant. From the confusion matrix, it was observed that the model was capable of classifying most images correctly with the maximum correctly identified images to be 951 out of 1000 for class 1 while class 6 suffered with the most misclassified images with 544 images classified correctly whereas 156, 4, 75, 49, 151, 2, 18 and 1 images were classified as class 0, class 1, class 2, class 3, class 4, class 5, class 8, class 9 respectively. The classification report indicates an overall accuracy, macro average score and weighted average score of 0.81. The minimum precision, recall and f1-score were 0.60, 0.54 and 0.57 respectively for class 6 that of the best performing class was 0.95 for class 1. For the second model, the accuracy curve shows slight fluctuation in the values of accuracy after significant increase in the first few iterations. Despite the fluctuations after the first few iterations, the accuracy curve indicates a slow increase in accuracy values resulting in an accuracy of 0.85 by the end of the training process. The loss curve represents a considerable decrease in loss values in the initial iterations itself after which the rate of loss is rather slow with a significant number of fluctuations. The confusion matrix indicates that the model was able to

perform well for most of the classes with the 833, 958, 826, 897, 725, 910, 590, 883, 957 and 934 correctly classified instances out of 1000 images each for class 0, 1, 2, 3, 4, 5, 6, 7, 8 and 9 respectively. The highest amount of misclassification is observed for class 5 with 178 and 119 images misclassified images as class 0 and 2. Similarly, 135 images belonging to class 4 was misclassified as class 2. From the classification report, the accuracy, macro average and weighted average values are observed to be 0.80. It supports the results of confusion matrix with class 1 performing the best with precision, recall and f1-score of 0.96 while class 6 indicated comparatively poor performance with precision, recall and f1-score of 0.70, 0.59 and 0.64 respectively.

V. DISCUSSION AND ANALYSIS

Effect of increasing hidden layers:

Comparing the performance of the two models, the model with two hidden layers achieved a higher overall accuracy (0.8) compared to the model with one hidden layer (0.76). The accuracy curves reflect this improvement, with the two-layer model reaching a final accuracy of 0.8, while the single-layer model plateaued at 0.76. The loss curves also highlight a more stable reduction in loss for the two-layer model, decreasing from around 36 to 2, compared to the single-layer model, which saw a drop from 2.75 to approximately 0.9.

In terms of precision, recall, and F1-scores, the two-layer model outperformed the single-layer model, particularly for classes 1 and 8, which saw improved metrics ranging from 0.91 to 0.96 in the two-layer model versus 0.90 to 0.95 in the single-layer model. This suggests that adding more hidden layers enhances the model's ability to differentiate between classes and improves overall performance.

Both models exhibited challenges with class 6, which had the lowest performance metrics across the board, indicating that further optimization or additional model refinements might be needed to improve classification for this class.

Effect of Initialization Mechanism and Activation Functions:

Different initialization mechanisms and activation functions have a significant impact on the performance of the ANN model with two hidden layers.

The impact of different initialization mechanism is as follows:

Random Initialization: Models trained with this mechanism showed varied results based on activation functions. ReLU activation provided smooth accuracy and loss curves with an accuracy of 0.8 and minimal misclassifications. Tanh activation achieved a similar accuracy of 0.8 with smooth curves, while Sigmoid activation led to slower convergence and an accuracy of 0.76, particularly struggling with class 6.

Kaiming Initialization: Models demonstrated more stable performance with this initialization. The ReLU activation function resulted in an accuracy of 0.83 with notable fluctuations during training. Tanh activation provided the highest accuracy of 0.85, though class 6 still faced issues. Sigmoid activation showed a moderate accuracy of 0.72, with significant challenges in classifying class 6.

Xavier Initialization: This initialization mechanism led to generally high performance. ReLU activation resulted in an accuracy of 0.84, showing strong classification but with some misclassifications for class 6. Tanh activation also achieved an accuracy of 0.84 with good performance overall, while Sigmoid activation yielded an accuracy of 0.72, with significant difficulties in classifying class 6.

The impact of different activation functions is as follows:

ReLU Activation: ReLU generally delivers good performance, achieving up to 0.84 accuracy. However, its loss curve often shows fluctuations because ReLU can lead to large, unstable gradients, especially in the early stages of training. This instability occurs because ReLU outputs either zero or a positive value, which can result in erratic weight updates and a zigzag pattern in the loss curve as the model adjusts to the data.

Tanh Activation: Tanh provides the best performance, with accuracy reaching up to 0.85 and a smooth loss curve. This smoothness is due to Tanh's output range of -1 to 1, which helps maintain more consistent gradients during training. The bounded gradient of the Tanh function ensures stable weight updates, leading to gradual, steady improvements in both accuracy and loss.

Sigmoid Activation: Sigmoid results in slower convergence, with a maximum accuracy of 0.76. This slower learning is due to the vanishing gradient problem, where the gradients become very small during backpropagation. Output range between 0 and 1 also leads to non-zero-centered

outputs, causing inefficient weight updates and contributing to the gradual increase in accuracy and slower overall convergence.

Effect of mitigating overfitting (using Dropout)

Applying dropout to both models effectively mitigated overfitting, leading to improved generalization. For the Random Initialization with ReLU activation, dropout resulted in an accuracy of 0.81 and a loss curve that stabilized after an initial steep decline, indicating reduced overfitting. The confusion matrix and classification report highlighted strong performance for class 1, though class 6 had notable misclassification. In the Kaiming Initialization with Hyperbolic Tangent activation model, dropout enhanced performance further, achieving an accuracy of 0.85 with a more stable loss curve and fewer fluctuations. This model exhibited robust classification across most classes, with improved results for class 6 compared to the non-dropout scenario. Overall, dropout effectively reduced overfitting and enhanced model performance in both cases.

VI. CONCLUSION

In this lab report, we investigated the impact of increasing hidden layers, initialization mechanisms, activation functions, and dropout on neural network performance for the Fashion MNIST dataset. Adding a second hidden layer improved accuracy to 0.8, compared to 0.76 with a single layer, demonstrating that deeper networks capture more complex patterns and offer enhanced performance, though class 6 remained challenging. Various initialization mechanisms and activation functions also influenced performance; Tanh with Kaiming Initialization yielded the highest accuracy of 0.85 due to its smooth gradient updates, while ReLU showed good performance but with fluctuations, and Sigmoid resulted in slower convergence. Implementing dropout effectively mitigated overfitting, improving accuracy and stability across both models, with the Kaiming Initialization and Tanh model achieving the best overall results and better handling class 6. These findings highlight the importance of network depth, proper initialization, activation functions, and dropout in optimizing neural network performance and generalization.

APPENDIX

A. Data Distribution



Figure 1 Class distribution of Fashion MNIST dataset for training data

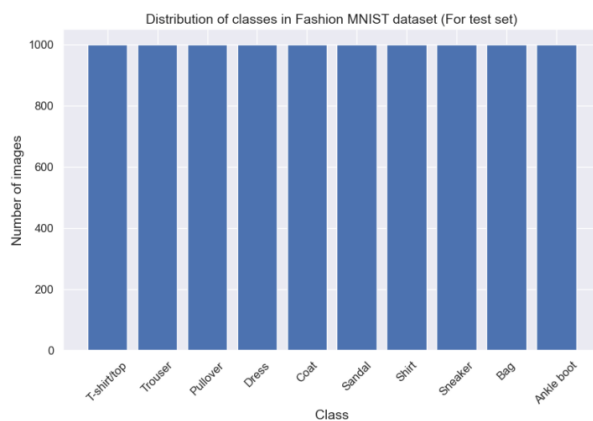


Figure 2 Class distribution for Fashion MNIST dataset for test data

B. Effects of Increasing Hidden Layers

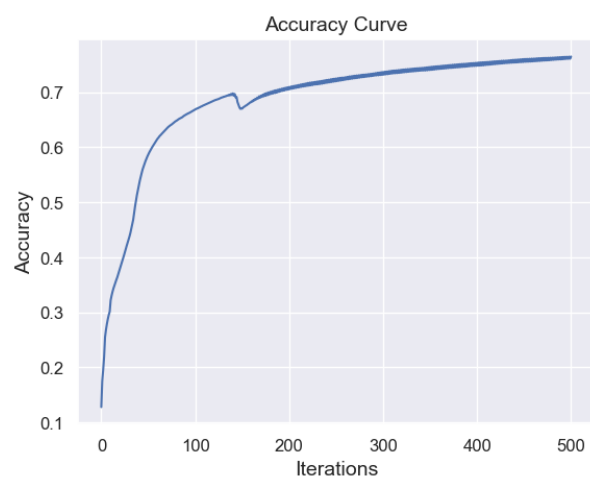


Figure 3 Accuracy Curve for ANN with 1 hidden layer using Random Initialization using ReLU and Softmax activation functions

A. Shrestha and P. Dongol: Artificial Neural Network

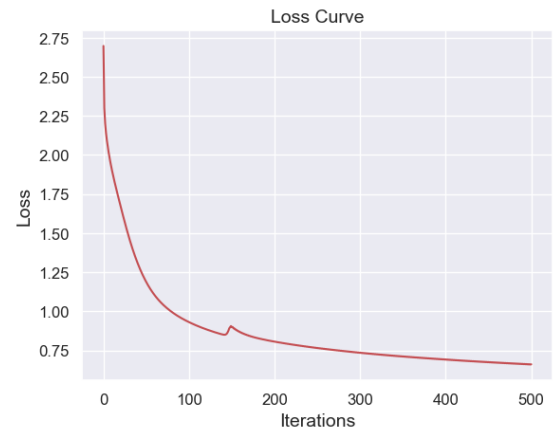


Figure 4 Loss Curve for ANN with 1 hidden layer using Random Initialization using ReLU and Softmax activation functions

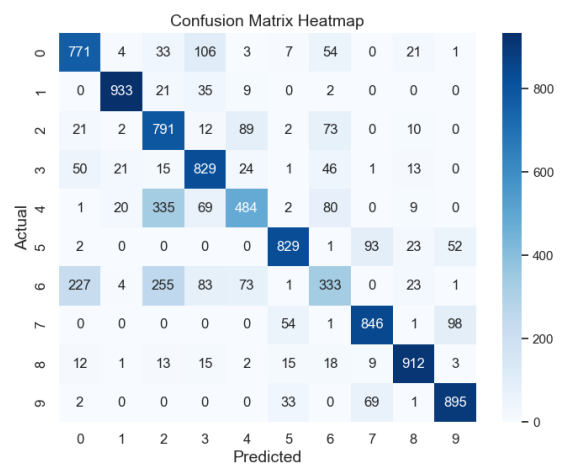


Figure 5 Confusion Matrix for ANN with 1 hidden layer using Random Initialization using ReLU and Softmax activation functions

Table 1 Classification Report for ANN with 1 hidden layer using Random Initialization using ReLU and Softmax activation functions

Class	Precision	Recall	F1-score	Support
0	0.71	0.77	0.74	1000
1	0.95	0.93	0.94	1000
2	0.54	0.79	0.64	1000
3	0.72	0.83	0.77	1000
4	0.71	0.48	0.57	1000
5	0.88	0.83	0.85	1000
6	0.55	0.33	0.41	1000
7	0.83	0.85	0.84	1000
8	0.90	0.91	0.91	1000
9	0.85	0.90	0.87	1000
Accuracy			0.76	10000
Macro avg	0.76	0.76	0.76	10000
Weighted avg	0.76	0.76	0.76	10000

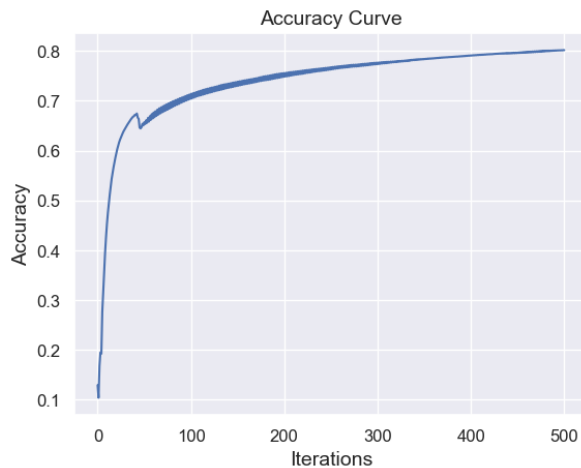


Figure 6 Accuracy Curve for ANN with 2 hidden layer using Random Initialization using ReLU and Softmax activation functions

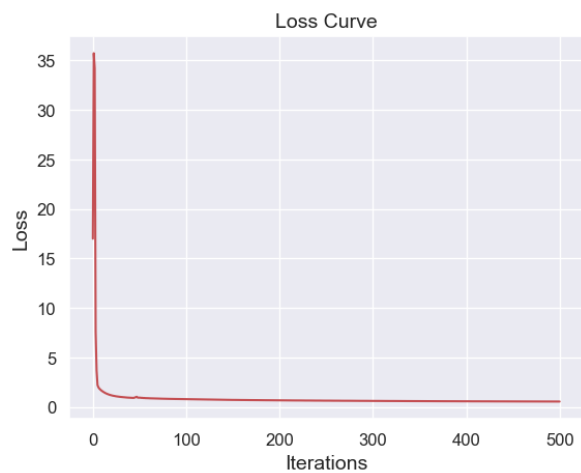


Figure 7 Loss Curve for ANN with 2 hidden layer using Random Initialization using ReLU and Softmax activation functions

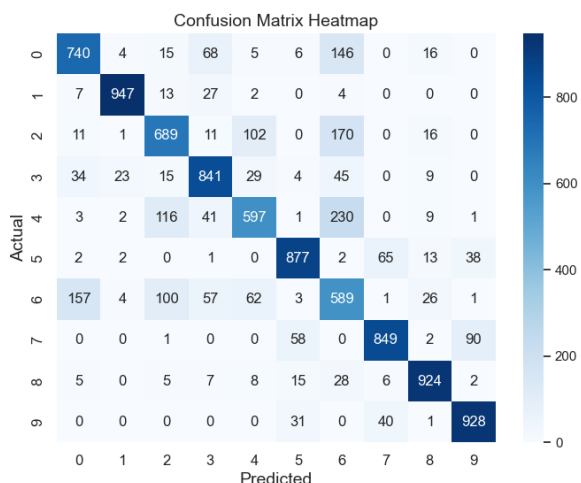


Figure 8 Confusion Matrix for ANN with 2 hidden layer using Random Initialization using ReLU and Softmax activation functions

Table 2 Classification Report for ANN with 2 hidden layer using Random Initialization using ReLU and Softmax activation functions

Class	Precision	Recall	F1-score	Support
0	0.77	0.74	0.76	1000
1	0.96	0.95	0.96	1000
2	0.72	0.69	0.71	1000
3	0.80	0.84	0.82	1000
4	0.74	0.60	0.66	1000
5	0.88	0.88	0.88	1000
6	0.49	0.59	0.53	1000
7	0.88	0.85	0.87	1000
8	0.91	0.92	0.92	1000
9	0.88	0.93	0.90	1000
Accuracy			0.80	10000
Macro avg	0.80	0.80	0.80	10000
Weighted avg	0.80	0.80	0.80	10000

C. Effects of Initialization Mechanisms and Activation Functions

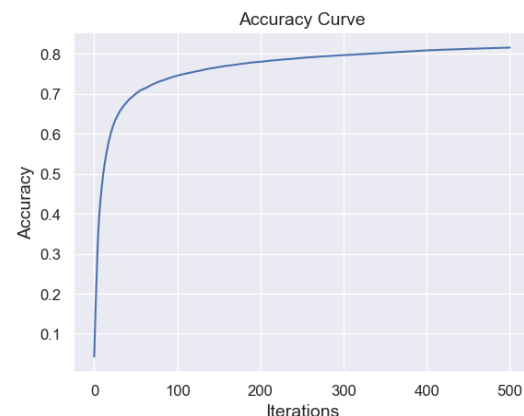


Figure 9 Accuracy Curve for ANN using Random Initialization using Hyperbolic Tangent and Softmax activation functions

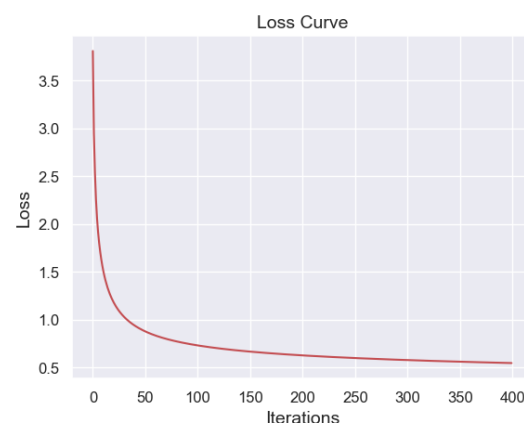


Figure 10 Loss Curve for ANN using Random Initialization using Hyperbolic Tangent and Softmax activation functions

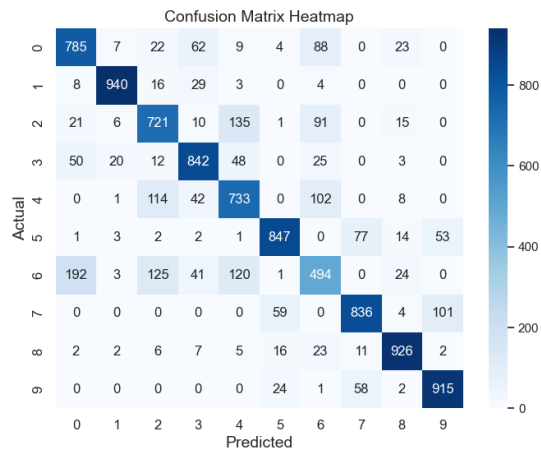


Figure 11 Confusion Matrix for ANN using Random Initialization using Hyperbolic Tangent and Softmax activation functions

Table 3 Classification Report for ANN using Random Initialization using Hyperbolic Tangent and Softmax activation functions

Class	Precision	Recall	F1-score	Support
0	0.74	0.79	0.76	1000
1	0.96	0.94	0.95	1000
2	0.71	0.72	0.71	1000
3	0.81	0.84	0.83	1000
4	0.70	0.73	0.71	1000
5	0.89	0.85	0.87	1000
6	0.60	0.49	0.54	1000
7	0.85	0.84	0.84	1000
8	0.91	0.93	0.92	1000
9	0.85	0.92	0.88	1000
Accuracy			0.80	10000
Macro avg	0.80	0.80	0.80	10000
Weighted avg	0.80	0.80	0.80	10000

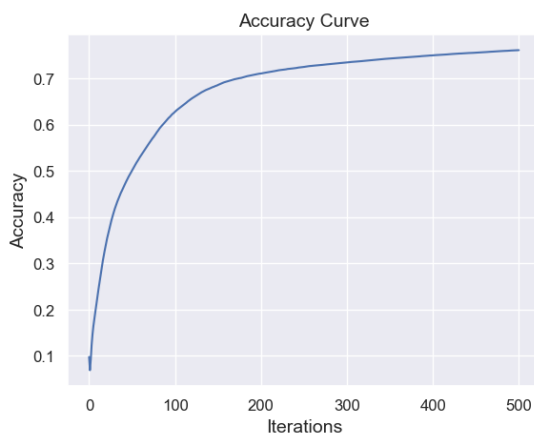


Figure 12 Accuracy Curve for ANN using Random Initialization using Sigmoid and Softmax activation functions

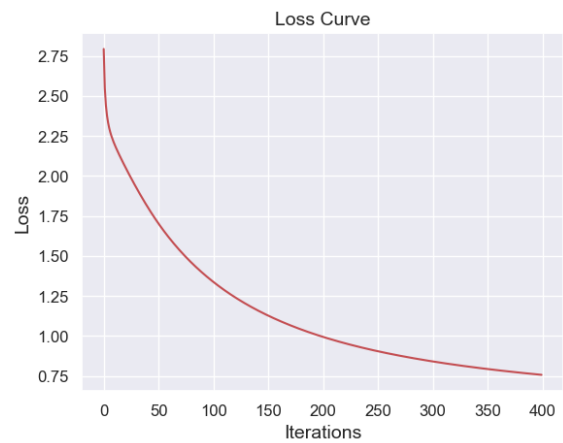


Figure 13 Loss Curve for ANN using Random Initialization using Sigmoid and Softmax activation functions

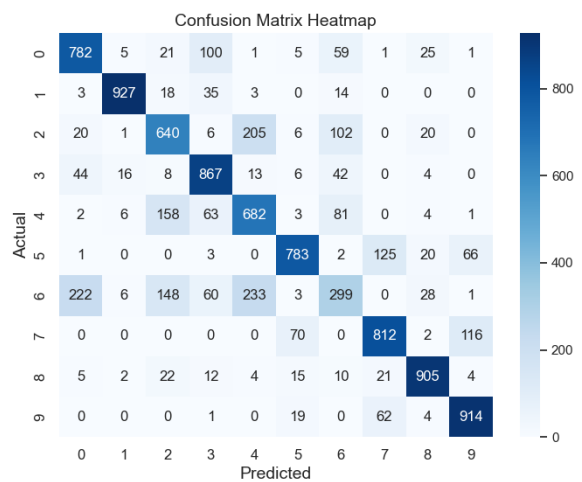


Figure 14 Confusion Matrix for ANN using Random Initialization using Sigmoid and Softmax activation functions

Table 4 Classification Report for ANN using Random Initialization using Sigmoid and Softmax activation functions

Class	Precision	Recall	F1-score	Support
0	0.72	0.78	0.75	1000
1	0.96	0.93	0.94	1000
2	0.63	0.64	0.64	1000
3	0.76	0.87	0.81	1000
4	0.60	0.68	0.64	1000
5	0.86	0.78	0.82	1000
6	0.49	0.30	0.37	1000
7	0.80	0.81	0.80	1000
8	0.89	0.91	0.90	1000
9	0.83	0.91	0.87	1000
Accuracy			0.76	10000
Macro avg	0.75	0.76	0.75	10000
Weighted avg	0.75	0.76	0.75	10000

Table 5 Classification Report for ANN using Kaiming Initialization using ReLU and Softmax activation functions

Class	Precision	Recall	F1-score	Support
0	0.71	0.89	0.79	1000
1	0.96	0.97	0.96	1000
2	0.62	0.88	0.72	1000
3	0.84	0.88	0.86	1000
4	0.80	0.68	0.73	1000
5	0.93	0.91	0.92	1000
6	0.84	0.34	0.48	1000
7	0.88	0.92	0.90	1000
8	0.94	0.95	0.95	1000
9	0.94	0.93	0.93	1000
Accuracy			0.83	10000
Macro avg	0.85	0.83	0.83	10000
Weighted avg	0.85	0.83	0.83	10000

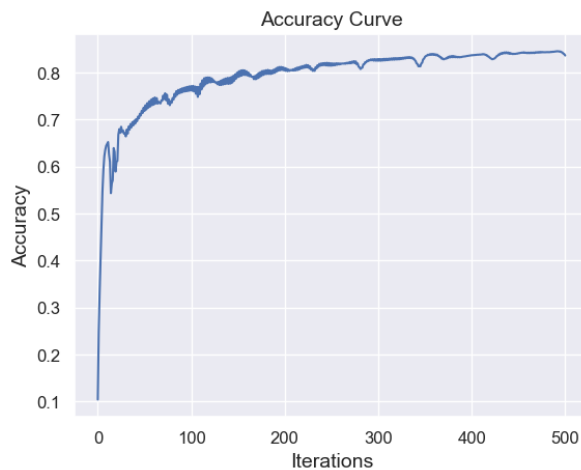


Figure 15 Accuracy Curve for ANN using Kaiming Initialization using ReLU and Softmax activation functions

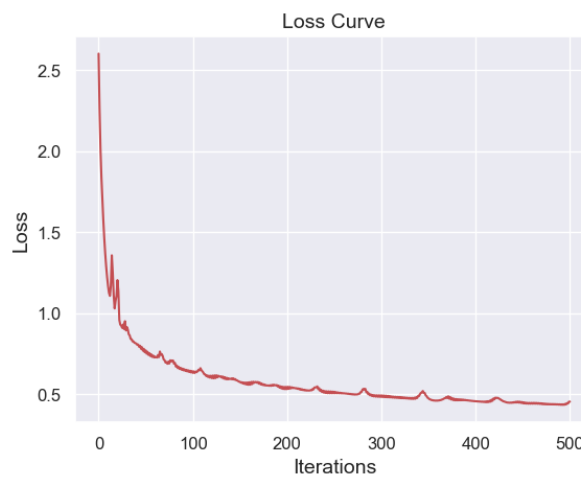


Figure 16 Loss Curve for ANN using Kaiming Initialization using ReLU and Softmax activation functions

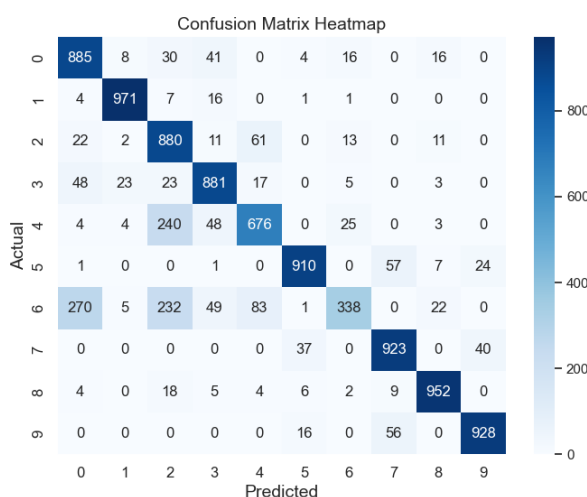


Figure 17 Confusion Matrix for ANN using Kaiming Initialization using ReLU and Softmax activation functions

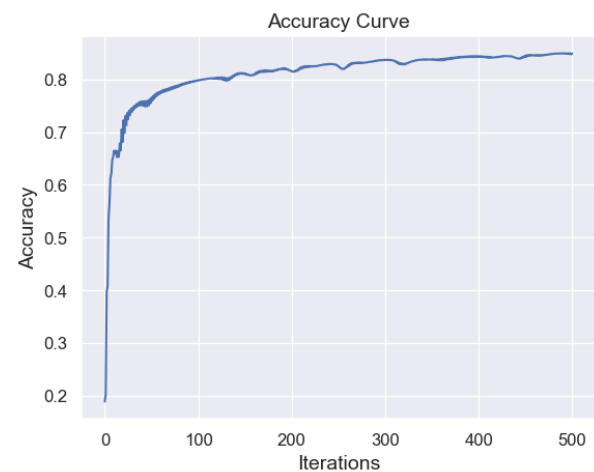


Figure 18 Accuracy Curve for ANN using Kaiming Initialization using Hyperbolic Tangent and Softmax activation functions

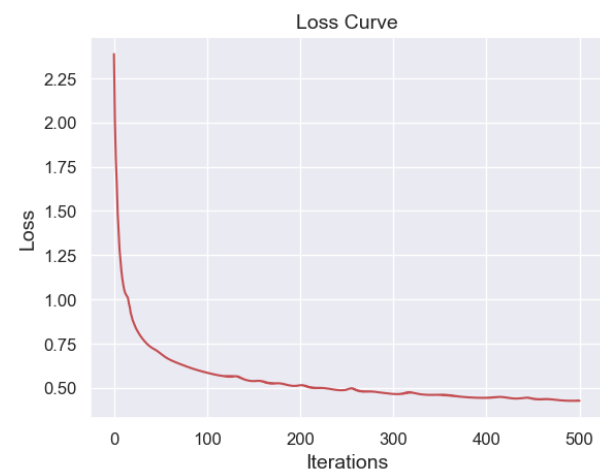


Figure 19 Loss Curve for ANN using Kaiming Initialization using Hyperbolic Tangent and Softmax activation functions

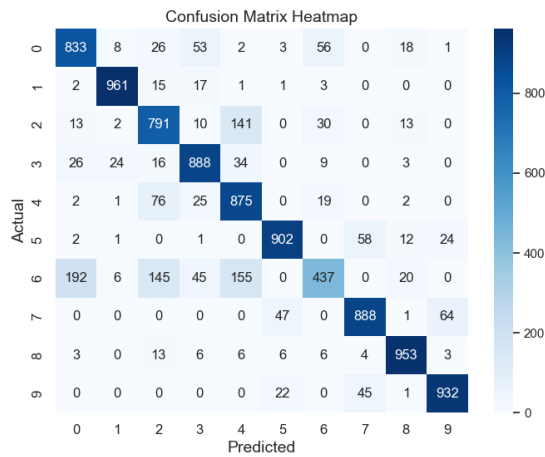


Figure 20 Confusion Matrix for ANN using Kaiming Initialization using Hyperbolic Tangent and Softmax activation functions

Table 6 Classification Report for ANN using Kaiming Initialization using Hyperbolic Tangent and Softmax activation functions

Class	Precision	Recall	F1-score	Support
0	0.78	0.83	0.80	1000
1	0.96	0.96	0.96	1000
2	0.73	0.79	0.76	1000
3	0.85	0.89	0.87	1000
4	0.72	0.88	0.79	1000
5	0.92	0.90	0.91	1000
6	0.78	0.44	0.56	1000
7	0.89	0.89	0.89	1000
8	0.93	0.95	0.94	1000
9	0.91	0.93	0.92	1000
Accuracy			0.85	10000
Macro avg	0.85	0.85	0.84	10000
Weighted avg	0.85	0.85	0.84	10000

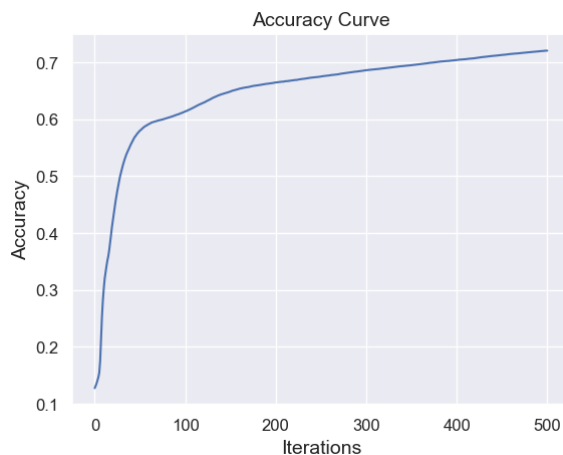


Figure 21 Accuracy Curve for ANN using Kaiming Initialization using Sigmoid and Softmax activation functions

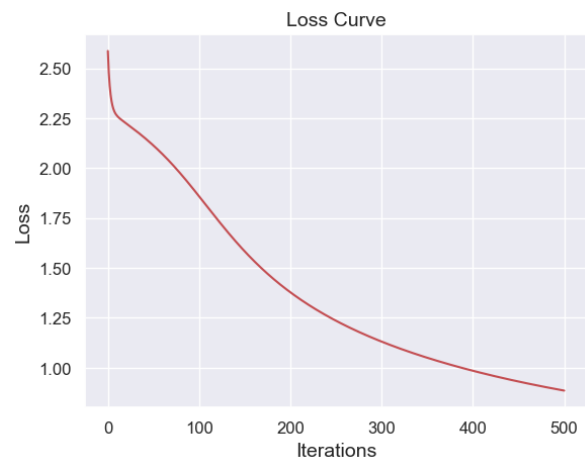


Figure 22 Loss Curve for ANN using Kaiming Initialization using Sigmoid and Softmax activation functions

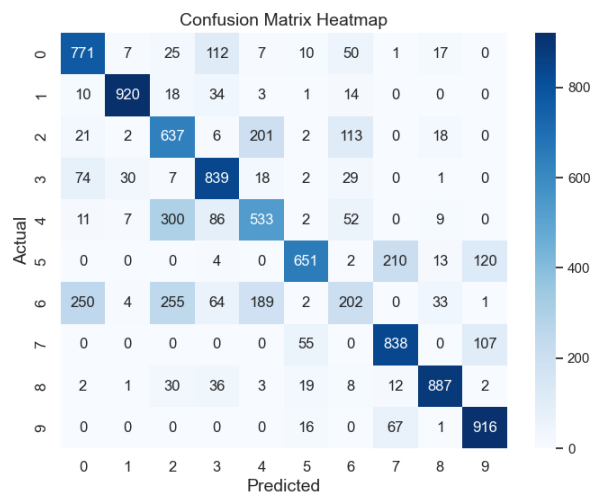


Figure 23 Confusion Matrix for ANN using Kaiming Initialization using Sigmoid and Softmax activation functions

Table 7 Classification Report for ANN using Kaiming Initialization using Sigmoid and Softmax activation functions

Class	Precision	Recall	F1-score	Support
0	0.68	0.77	0.72	1000
1	0.95	0.92	0.93	1000
2	0.50	0.64	0.56	1000
3	0.71	0.84	0.77	1000
4	0.56	0.53	0.55	1000
5	0.86	0.65	0.74	1000
6	0.43	0.20	0.27	1000
7	0.74	0.84	0.79	1000
8	0.91	0.89	0.90	1000
9	0.80	0.92	0.85	1000
Accuracy			0.72	10000
Macro avg	0.71	0.72	0.71	10000
Weighted avg	0.71	0.72	0.71	10000

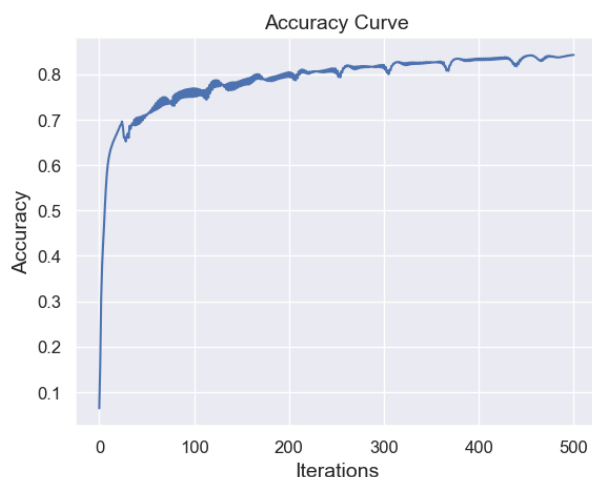


Figure 24 Accuracy Curve for ANN using Xavier Initialization using ReLU and Softmax activation functions

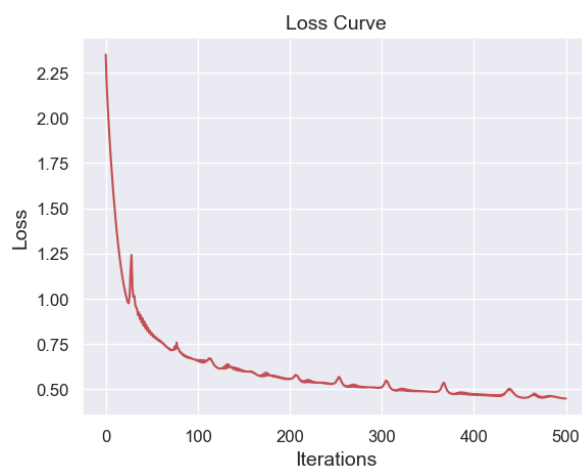


Figure 25 Loss Curve for ANN using Xavier Initialization using ReLU and Softmax activation functions

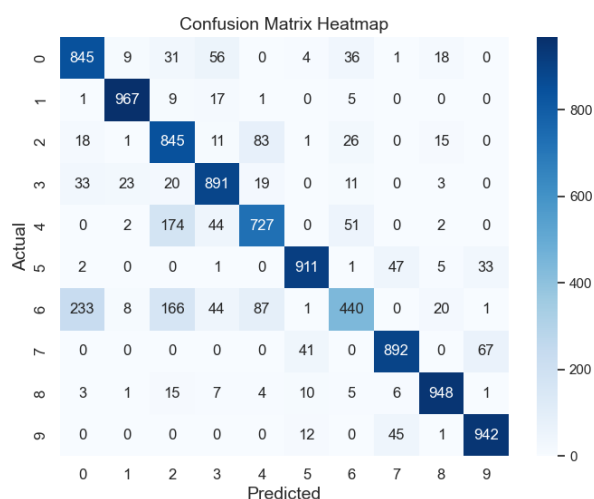


Figure 26 Confusion Matrix for ANN using Xavier Initialization using ReLU and Softmax activation functions

Table 8 Classification Report for ANN using Xavier Initialization using ReLU and Softmax activation functions

Class	Precision	Recall	F1-score	Support
0	0.74	0.84	0.79	1000
1	0.96	0.97	0.96	1000
2	0.67	0.84	0.75	1000
3	0.83	0.89	0.86	1000
4	0.79	0.73	0.76	1000
5	0.93	0.91	0.92	1000
6	0.77	0.44	0.56	1000
7	0.90	0.89	0.90	1000
8	0.94	0.95	0.94	1000
9	0.90	0.94	0.92	1000
Accuracy			0.84	10000
Macro avg	0.84	0.84	0.84	10000
Weighted avg	0.84	0.84	0.84	10000

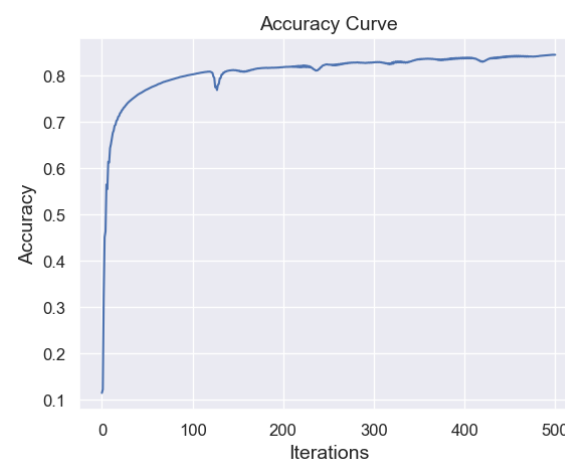


Figure 27 Accuracy Curve for ANN using Xavier Initialization using Hyperbolic Tangent and Softmax activation functions

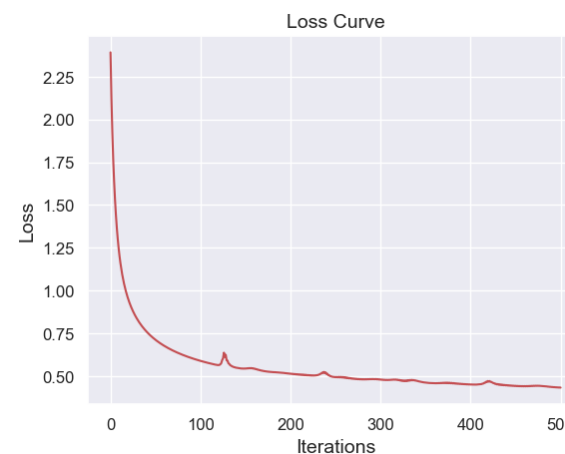


Figure 28 Loss Curve for ANN using Xavier Initialization using Hyperbolic Tangent and Softmax activation functions

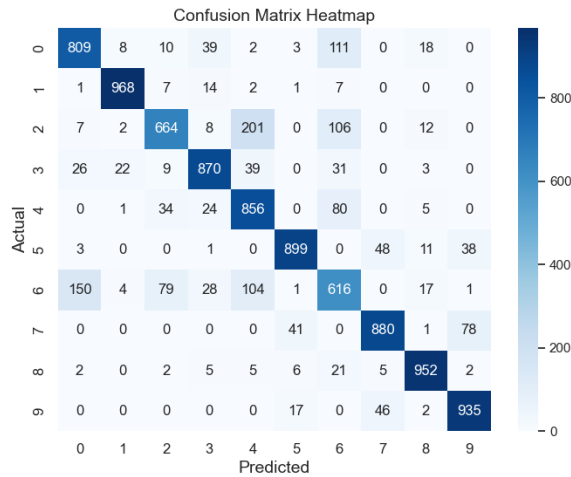


Figure 29 Confusion Matrix for ANN using Xavier Initialization using Hyperbolic Tangent and Softmax activation functions

Table 9 Classification Report for ANN using Xavier Initialization using Hyperbolic Tangent and Softmax activation functions

Class	Precision	Recall	F1-score	Support
0	0.81	0.81	0.81	1000
1	0.96	0.97	0.97	1000
2	0.82	0.66	0.74	1000
3	0.88	0.87	0.87	1000
4	0.71	0.86	0.78	1000
5	0.93	0.90	0.91	1000
6	0.63	0.62	0.62	1000
7	0.90	0.88	0.89	1000
8	0.93	0.95	0.94	1000
9	0.89	0.94	0.91	1000
Accuracy			0.84	10000
Macro avg	0.85	0.84	0.84	10000
Weighted avg	0.85	0.84	0.84	10000

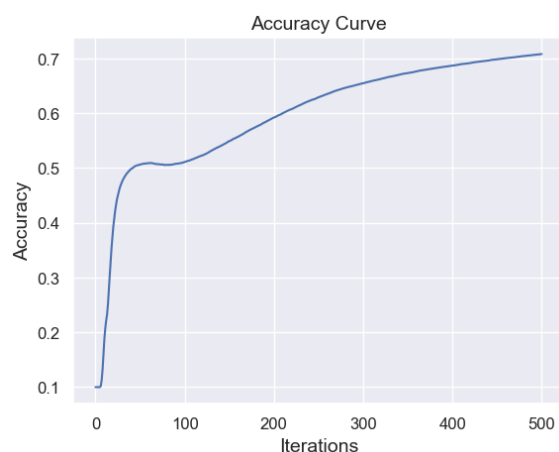


Figure 30 Accuracy Curve for ANN using Xavier Initialization using Sigmoid and Softmax activation functions

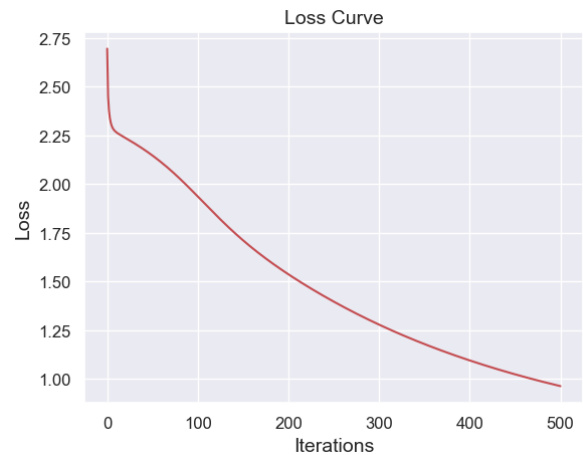


Figure 31 Loss Curve for ANN using Xavier Initialization using Sigmoid and Softmax activation functions

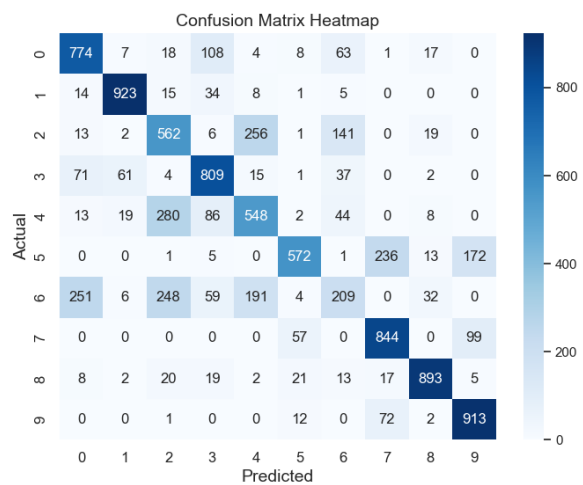


Figure 32 Confusion Matrix for ANN using Xavier Initialization using Sigmoid and Softmax activation functions

Table 10 Classification Report for ANN using Xavier Initialization using Sigmoid and Softmax activation functions

Class	Precision	Recall	F1-score	Support
0	0.68	0.77	0.72	1000
1	0.90	0.92	0.91	1000
2	0.49	0.56	0.52	1000
3	0.72	0.81	0.76	1000
4	0.54	0.55	0.54	1000
5	0.84	0.57	0.68	1000
6	0.41	0.21	0.28	1000
7	0.72	0.84	0.78	1000
8	0.91	0.89	0.90	1000
9	0.77	0.91	0.83	1000
Accuracy			0.70	10000
Macro avg	0.70	0.70	0.69	10000
Weighted avg	0.70	0.70	0.69	10000

D. Mitigating Overfitting

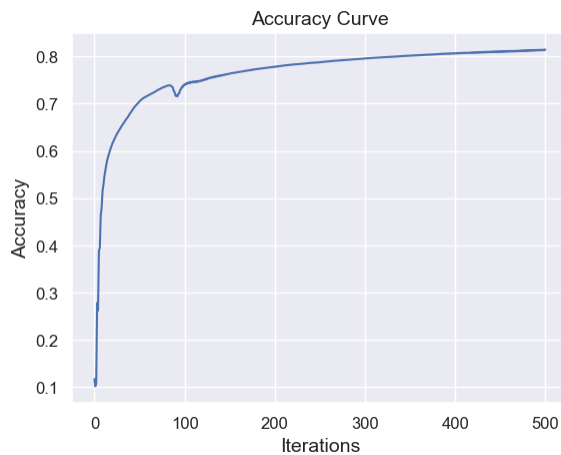


Figure 33 Accuracy Curve for ANN using dropouts using Random Initialization with ReLU and Softmax activation functions

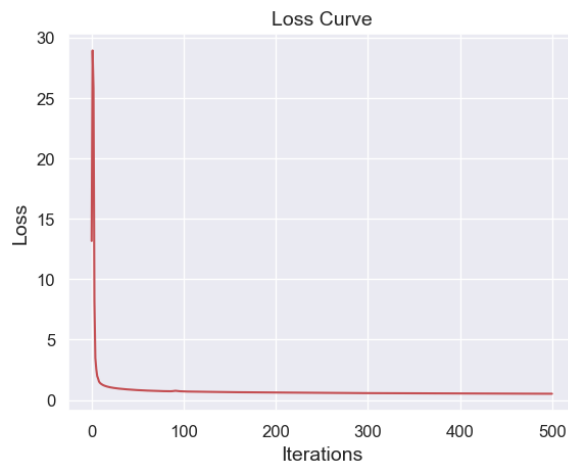


Figure 34 Loss Curve for ANN using dropouts using Random Initialization with ReLU and Softmax activation functions

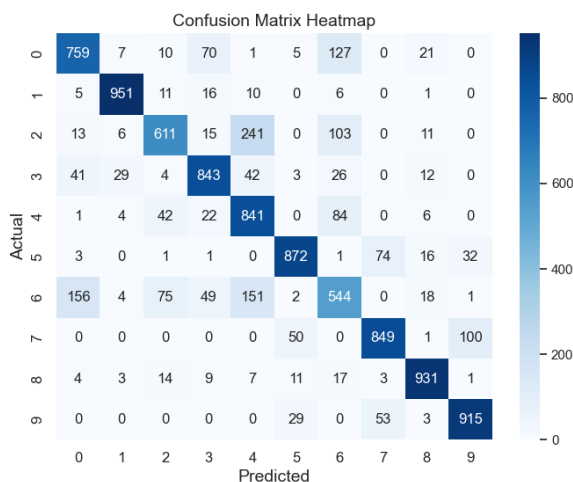


Figure 35 Confusion Matrix for ANN using dropouts using Random Initialization with ReLU and Softmax activation functions

A. Shrestha and P. Dongol: Artificial Neural Network

Table 11 Classification Report for ANN using dropouts using Random Initialization with ReLU and Softmax activation functions

Class	Precision	Recall	F1-score	Support
0	0.77	0.76	0.77	1000
1	0.95	0.95	0.95	1000
2	0.80	0.61	0.69	1000
3	0.82	0.84	0.83	1000
4	0.65	0.84	0.73	1000
5	0.90	0.87	0.88	1000
6	0.60	0.54	0.57	1000
7	0.87	0.85	0.86	1000
8	0.91	0.93	0.92	1000
9	0.87	0.92	0.89	1000
Accuracy			0.81	10000
Macro avg	0.81	0.81	0.81	10000
Weighted avg	0.81	0.81	0.81	10000

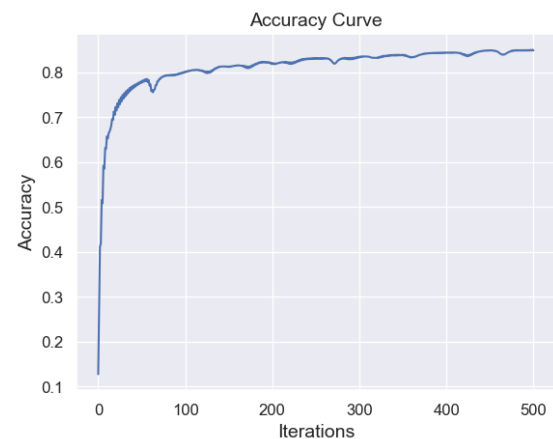


Figure 36 Accuracy Curve for ANN using dropouts using Kaiming Initialization with Hyperbolic Tangent and Softmax activation functions

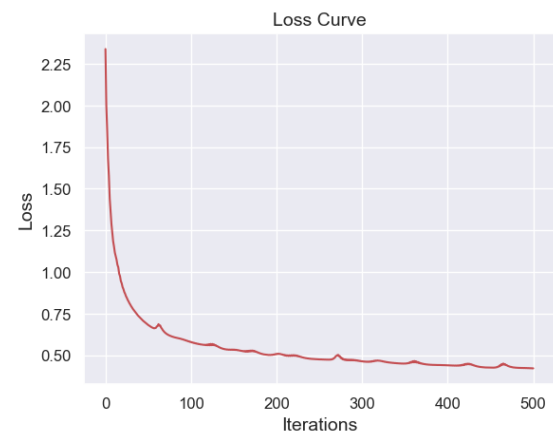


Figure 37 Loss Curve for ANN using dropouts using Kaiming Initialization with Hyperbolic Tangent and Softmax activation functions

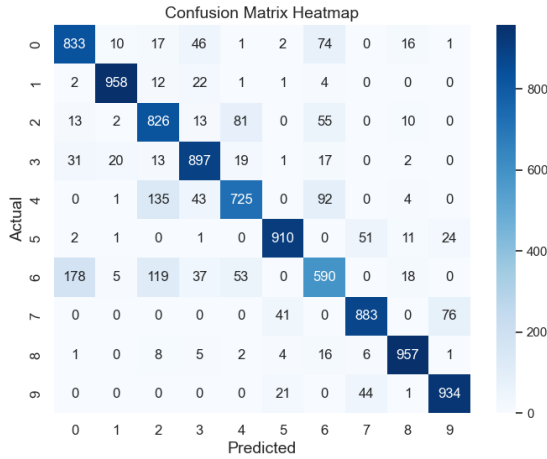


Figure 38 Confusion Matrix for ANN using dropouts using Kaiming Initialization with Hyperbolic Tangent and Softmax activation functions

Table 12 Classification Report for ANN using dropouts using Random Initialization with ReLU and Softmax activation functions

Class	Precision	Recall	F1-score	Support
0	0.79	0.83	0.81	1000
1	0.96	0.96	0.96	1000
2	0.73	0.83	0.78	1000
3	0.84	0.90	0.87	1000
4	0.82	0.72	0.77	1000
5	0.93	0.91	0.92	1000
6	0.70	0.59	0.64	1000
7	0.90	0.88	0.89	1000
8	0.94	0.96	0.95	1000
9	0.90	0.93	0.92	1000
Accuracy			0.85	10000
Macro avg	0.85	0.85	0.85	10000
Weighted avg	0.85	0.85	0.85	10000

E. Derivations

To derive the gradient of the weights with respect to the cost function, the equations for forward propagation are

For layer 1,

$$Z_1 = W_1 A_0 + b_1$$

$$A_1 = \phi(Z_1)$$

For layer 2,

$$Z_2 = W_2 A_1 + b_2$$

$$A_2 = \text{softmax}(Z_2)$$

Assume that the cost function J is the cross-entropy loss for classification tasks. The derivative of the cost function with respect to Z_2 is denoted as dZ_2 .

A. Shrestha and P. Dongol: Artificial Neural Network

The gradient of cost function with respect to Z_2 is

$$dZ_2 = A_2 - Y$$

where, Y is the true label vector

Gradients of weight ' W_2 ':

The gradient of cost function with respect to W_2 is

$$\frac{\partial J}{\partial W_2} = \frac{\partial J}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial W_2}$$

where,

$$\frac{\partial Z_2}{\partial W_2} = A_1$$

So,

$$\frac{\partial J}{\partial W_2} = dZ_2 \cdot A_1^T$$

where, dZ_2 is the error term for layer 2 and A_1^T is the transpose of the activation values from layer 1

Taking into account that dZ_2 is the error term for layer 2, we average over all training examples

$$dW_2 = \frac{1}{m} \cdot dZ_2 \cdot A_1^T$$

Thus, the gradient of cost function with respect to W_2 is

$$dW_2 = \frac{1}{m} \sum_{i=1}^m dZ_2^{(i)} \cdot (A_1^{(i)})^T$$

where, m is the number of training examples

Gradient of the bias term ' b_2 '

The gradient of cost function with respect to bias term is

$$\frac{\partial J}{\partial b_2} = 1$$

Applying chain rule,

$$\frac{\partial J}{\partial b_2} = \frac{\partial J}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial b_2}$$

Then

$$\frac{\partial J}{\partial W_2} = dZ_2 \cdot 1$$

Averaging over all training examples,

$$db_2 = \frac{1}{m} \sum_{i=1}^m dZ_2^{(i)}$$

Gradient of Cost Function with Respect to ' Z_1 '

The gradient of cost function with respect to dZ_1 is

$$dZ_1 = \frac{\partial J}{\partial Z_1}$$

Computing using dZ_2 ,

$$dZ_1 = W_2^T dZ_2 \cdot \phi'(Z_1)$$

where, $\phi'(Z_1)$ represents the derivative of the activation function

Gradient with Respect to weight 'W₁'

The weight gradient dW_1 can be derived from

$$\frac{\partial J}{\partial W_1} = \frac{\partial J}{\partial Z_1} \cdot \frac{\partial Z_1}{\partial W_1}$$

We have,

$$\frac{\partial Z_1}{\partial W_1} = A_0$$

So,

$$dW_1 = \frac{1}{m} \sum_{i=1}^m dZ_1^{(i)} \cdot (A_0^{(i)})^T$$

Gradient with Respect to Biases 'b₁'

The bias gradient db_1

$$\frac{\partial J}{\partial b_1} = \frac{\partial J}{\partial Z_1} \cdot \frac{\partial Z_1}{\partial b_1}$$

Since,

$$\frac{\partial Z_1}{\partial b_1} = 1$$

So,

$$db_1 = \frac{1}{m} \sum_{i=1}^m dZ_1^{(i)}$$

Derivative of Loss Function:

The cross-entropy is given by

$$L = - \sum_{j=1}^K \hat{y}_j \log(\hat{y}_j)$$

The derivative of loss with respect to $\hat{y}_j$ is

$$\frac{\partial J}{\partial \hat{y}_j} = -\frac{y_j}{\hat{y}_j}$$

The derivative of $\hat{y}_j$ with respect to z_j is

When $i = j$

$$\frac{\partial \hat{y}_i}{\partial z_j} = \hat{y}_j(1 - \hat{y}_j)$$

When $i \neq j$

$$\frac{\partial \hat{y}_i}{\partial z_j} = \hat{y}_i \hat{y}_j$$

Using Kronecker delta function δ_{ij} which is 1 if $i = j$ and 0 otherwise

A. Shrestha and P. Dongol: Artificial Neural Network

$$\frac{\partial \hat{y}_i}{\partial z_j} = \hat{y}_j(\delta_{ij} - \hat{y}_j)$$

Combining the derivatives using the chain rule

$$\frac{\partial L}{\partial z_j} = \sum_{i=1}^K \frac{\partial L}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial z_j}$$

$$\frac{\partial L}{\partial z_j} = \sum_{i=1}^K \left(-\frac{y_j}{\hat{y}_j}\right) \hat{y}_i(\delta_{ij} - \hat{y}_j)$$

Since y_i is 0 for all $i \neq j$, the sum simplifies

$$\frac{\partial L}{\partial z_j} = \left(-\frac{y_j}{\hat{y}_j}\right) \hat{y}_j(1 - \hat{y}_j) + \sum_{i \neq j}^K \left(-\frac{y_j}{\hat{y}_j}\right) \hat{y}_i(-\hat{y}_j)$$

$$\frac{\partial L}{\partial z_j} = \left(-\frac{y_j}{\hat{y}_j}\right) \hat{y}_j(1 - \hat{y}_j) + \sum_{i \neq j}^K 0$$

$$\frac{\partial L}{\partial z_j} = -\hat{y}_j(1 - \hat{y}_j)$$

Since $\sum_{i \neq j}^K 0$ is 0,

$$\frac{\partial L}{\partial z_j} = \hat{y}_j - y_j$$

F. System Architecture

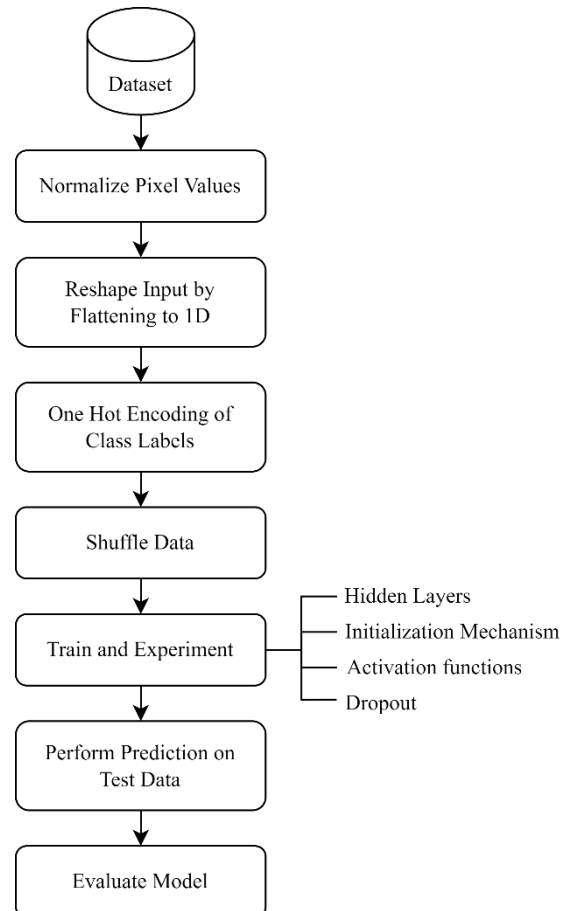


Figure 39 System Architecture

ACKNOWLEDGEMENT

We extend our heartfelt appreciation to the Department of Electronics and Computer Engineering at IOE, Thapathali Campus for providing us with the invaluable opportunity to engage in learning, research, and the realization of our insights. We would like to express our sincere gratitude to our esteemed lecturer Er. Dinesh Baniya Kshatri for providing unwavering support, invaluable guidance, and insightful suggestions throughout our academic career.

REFERENCES

- [1] H. Xiao, K. Rasul and R. Vollgraf, "Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms," *arXiv preprint*, 2017.
- [2] Y. Li, Y. Wang and Y. Zhang, "A Simple Feedforward Neural Network for Fashion MNIST Classification," *Journal of Machine Learning Research*, vol. 18, no. 1, pp. 1-10, 2018.
- [3] J. Bergstra and Y. Bengio, "Random Search for Hyper-Parameter Optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281-305, 2012.
- [4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929-1958, 2014.
- [5] V. Nair and G. E. Hilton, "Rectified Linear Units Improve Restricted Boltzmann Machines," *Proceedings of the 27th International Conference on Machine Learning (ICML-10)*, pp. 807-814, 2010.
- [6] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," *arXiv preprint*, 2015.
- [7] X. Zhang, Y. Li and Y. Wang, "Improving Feedforward Neural Networks with Batch Normalization on Fashion MNIST," *IEEE Transactions on Neural Networks and*

A. Shrestha and P. Dongol: Artificial Neural Network

Learning Systems, vol. 28, no. 10, pp. 2195-2202, 2017.

BIOGRAPHY



Aayush Man Shrestha

is currently pursuing his undergraduate degree in Computer Engineering at the Institute of Engineering (IOE), Thapathali Campus. His research interests span across diverse areas of ML and AI, including natural language processing, computer vision, robotics, and automation.



Pradeepti Dongol

is currently pursuing her undergraduate degree in Computer Engineering at the Institute of Engineering (IOE), Thapathali Campus. Her research interests span across diverse areas, including natural language processing, generative AI, arts and design and Large Language Models (LLMs).